

8교시: Prometheus/Grafana observability preview

수업 목표

- `docker logs`, `docker stats`가 observability stack으로 어떻게 확장되는지 체험한다.
- Prometheus, Grafana, cAdvisor, Loki의 역할을 구분한다.
- metrics와 logs를 같은 장애 상황에서 서로 다른 증거로 해석한다.

개념 설명

`logs`, `exec`, `stats`만 배우면 명령어 목록처럼 느껴질 수 있다. 하지만 운영에서는 이 명령들이 dashboard, time series, log query로 확장된다. Day 4의 마지막 장면은 이 확장을 미리 보는 것이다.

깊은 observability 수업은 아니다. 목표는 다음 차이를 감각적으로 잡는 것이다.

`docker logs`: 지금 무슨 일이 있었는가
`docker exec`: container 안쪽을 직접 확인한다
`docker stats`: 지금 순간의 resource를 본다
Prometheus/Grafana: resource 변화의 흐름을 본다
Loki/Grafana: log를 query하고 탐색한다

Preview stack

Service	역할	확인 지점
sample-web	nginx test traffic 대상	<code>curl</code> , access log
log-generator	일반 log sample	<code>docker compose logs</code> , Loki
load-generator	반복 HTTP 요청 생성	nginx access 증가
cpu-spike	선택 profile, CPU spike 생성	<code>docker stats</code> , Prometheus query
cadvisor	선택 profile, Docker container metrics 노출	Prometheus target
prometheus	metrics scrape/query	up, CPU/memory query
loki	log 저장	Grafana Explore
promtail	선택 profile, Docker log 전달	Loki ingest
grafana	metrics/logs 탐색 UI	Explore

실습 명령

기본 실행은 host의 Docker data root를 mount하지 않는다. Docker Desktop, WSL, macOS 환경에서 `/var/lib/docker`가 read-only로 막히는 경우가 있기 때문이다.

```
cd /mnt/d/paperclip/week2/day4/labs/observability-preview
docker compose config
docker compose --profile load config
docker compose up -d
docker compose ps
for i in 1 2 3 4 5; do curl -I http://localhost:18085 || true; sleep 1; done
```

```
docker compose logs sample-web --tail 20
docker compose logs log-generator --tail 20
```

Expected:

```
HTTP/1.1 200 OK
level=info service=log-generator event=heartbeat
```

Open:

```
Grafana: http://localhost:13000 admin / practice-only
Prometheus: http://localhost:19090
```

주소를 헛갈리면 안 된다. 브라우저에서 보는 Prometheus 주소와 Grafana가 내부에서 붙는 Prometheus 주소는 다르다.

위치	올바른 주소	이유
내 PC browser/curl	http://localhost:19090	host port 19090이 Prometheus container의 9090으로 publish됨
Grafana datasource URL	http://prometheus:9090	Grafana container는 Compose network에서 service name과 container port로 접근함
잘못된 값	http://localhost:19090	Grafana container 안의 localhost는 Grafana 자기 자신임

Grafana에서 Data source를 직접 수정했다면 Prometheus URL은 반드시 http://prometheus:9090으로 둔다. 이미 localhost:19090으로 저장했다면 Grafana UI에서 고치거나, 실습 data를 버려도 되는 경우에만 docker compose down -v로 grafana-data volume을 초기화한다.

기본 실행의 성공 기준은 sample-web, log-generator, loki, prometheus, grafana가 올라오고 일반 로그를 확인하는 것이다.

선택 심화: cAdvisor/Promtail host mount

cAdvisor와 Promtail은 Docker engine의 내부 data root를 읽어야 해서 OS와 Docker Desktop 구성에 영향을 받는다. 성공하면 container metrics와 Docker log file 수집까지 볼 수 있지만, 실패해도 Day 4 preview 실패로 보지 않는다.

WSL/Linux에서 시도:

```
export DOCKER_ROOT_DIR="$(docker info --format '{{.DockerRootDir}}')"
docker compose --profile host-mount up -d cAdvisor promtail
docker compose ps
```

접속:

```
cAdvisor: http://localhost:18086
```

다음 예러가 나오면 host mount 제약이다.

```
Error response from daemon: error while creating mount source path '/var/lib/docker':
mkdir /var/lib/docker: read-only file system
```

이 예러는 container 내부 문제가 아니라 Docker engine이 bind mount source path를 만들거나 노출할 수 없는 환경이라는 뜻이다. 이 경우에는 cAdvisor, promtail을 포기하고 다음으로 대체한다.

```
docker compose logs sample-web --tail 20
docker compose logs log-generator --tail 20
docker stats --no-stream
```

Impact drill

CPU spike를 일부러 만들고 snapshot과 time series를 비교한다.

```
docker compose --profile load up -d cpu-spike
docker stats --no-stream
sleep 20
```

Grafana Explore > Prometheus:

```
rate(container_cpu_usage_seconds_total[1m])
```

해석:

관찰 도구	보이는 것	한계
docker stats --no-stream	지금 순간 CPU/MEM	이전 1분 동안의 추세를 보기 어렵다
Prometheus query	시간 window의 변화	어떤 log line 때문에 튀었는지는 별도 log 확인 필요
Grafana Explore	metrics와 logs를 한 화면에서 탐색	dashboard가 원인을 자동으로 고쳐주지는 않는다

CPU spike를 멈춘다.

```
docker compose stop cpu-spike
```

Grafana Explore 확인

Source	Query	의미
Prometheus	up	scrape 대상이 살아 있는지 확인
Prometheus	container_memory_usage_bytes	host-mount profile 성공 시 container memory metrics 확인
Prometheus	rate(container_cpu_usage_seconds_total[1m])	host-mount profile 성공 시 CPU 사용률 변화 확인
Loki	{job="docker"}	host-mount profile 성공 시 Docker container log 확인

Prometheus에서 go_*, prometheus_*, process_* metric만 보이고 container_cpu_usage_seconds_total이 보이지 않으면 Prometheus가 자기 자신만 scrape하고 있다는 뜻이다. container metric은 Prometheus가 직접 만드는 값이 아니라 cAdvisor가 /metrics로 노출하고 Prometheus가 scrape해야 생긴다.

먼저 target을 확인한다.

```
curl -s 'http://localhost:19090/api/v1/query?query=up'
```

정상 예시:

```
instance="prometheus:9090", job="prometheus", value=1
instance="cadvisor:8080", job="cadvisor", value=1
```

cadvisor가 없거나 0이면 container metric은 나오지 않는다. 이때는 선택 profile을 켜다.

```
export DOCKER_ROOT_DIR="$(docker info --format '{{.DockerRootDir}}')"
docker compose --profile host-mount up -d cadvisor
curl -s 'http://localhost:19090/api/v1/query?query=up'
```

cAdvisor만 죽어 있거나 Prometheus에서 network 오류가 보이면 다음 순서로 분리한다.

증상	의미	확인 명령	조치
cadvisor container가 Exited	cAdvisor 자체가 시작 실패	docker compose ps cadvisor, docker compose logs cadvisor --tail 80	mount/device 오류 확인
cadvisor는 Up인데 Prometheus target이 down	Prometheus가 cadvisor:8080을 같은 Compose network에서 못 찾음	curl -s http://localhost:19090/api/v1/targets	같은 directory/project에서 다시 실행
no such host cadvisor	Prometheus network 안에 cadvisor service DNS가 없음	docker compose ps cadvisor, docker network ls	docker compose --profile host-mount up -d cadvisor
connection refused	이름은 resolve됐지만 cAdvisor 8080이 열려 있지 않음	docker compose logs cadvisor --tail 80	cAdvisor 시작 실패 원인 확인
go_* metric만 보임	Prometheus 자기 자신만 scrape 중	curl -s 'http://localhost:19090/api/v1/query?query=up'	host-mount profile로 cAdvisor 실행
mkdir /var/lib/docker: read-only file system	Docker data root mount 불가	docker info --format '{{.DockerRootDir}}'	실제 DockerRootDir export 후 재시도
/dev/kmsg 또는 /dev/disk 오류	runtime/device mount 제약	docker compose logs cadvisor --tail 80	Docker Desktop/WSL 제약으로 보고 기본 preview로 진행

network 오류는 대개 cadvisor만 다른 project/network로 띄웠을 때 생긴다. Prometheus 설정은 cadvisor:8080이라는 Compose service name을 scrape하므로, docker run으로 따로 띄운 cAdvisor나 다른 디렉터리에서 뜬 cAdvisor는 같은 이름으로 resolve되지 않는다.

항상 같은 디렉터리에서 같은 project로 실행한다.

```
cd /mnt/d/paperclip/week2/day4/labs/observability-preview
docker compose up -d
export DOCKER_ROOT_DIR="$(docker info --format '{{.DockerRootDir}}')"
```

```
docker compose --profile host-mount up -d cadvisor promtail
docker compose ps
```

network가 맞는지 확인:

```
docker inspect observability-preview-prometheus-1 --format '{{json .NetworkSettings.Networks}}'
```

```
docker inspect observability-preview-cadvisor-1 --format '{{json .NetworkSettings.Networks}}'
```

둘 다 observability-preview_observe-net에 붙어 있어야 한다.

그 다음에야 다음 query가 의미를 가진다.

```
rate(container_cpu_usage_seconds_total[1m])
container_memory_usage_bytes
```

container별로 좁혀서 보고 싶으면 label filter를 쓴다.

기준	PromQL 예시	언제 쓰는가		
container name	container_memory_usage_bytes{name=~"observability-preview-grafana-1"}	실제 container 이름으로 정확히 찾을 때		
container name 패턴	container_memory_usage_bytes{name=~".*grafana.*"}	이름 일부만 기억할 때		
Compose service	container_memory_usage_bytes{container_label_com_docker_compose_service="grafana"}	Compose service 단위로 볼 때		
여러 service	`container_memory_usage_bytes{container_label_com_docker_compose_service=~"grafana`	prometheus	loki}`	관찰 stack 일부만 볼 때

CPU도 같은 방식으로 필터링한다.

```
rate(container_cpu_usage_seconds_total{name=~".*cpu-spike.*"}[1m])
```

수업에서는 name보다 container_label_com_docker_compose_service를 먼저 권장한다. container name은 project name과 번호가 붙어서 바꿀 수 있지만, Compose service name은 실습 구조를 설명하기 좋다.

차트 legend가 너무 지저분하면 query가 아니라 panel의 Legend 값을 바꾼다.

원하는 legend	Grafana Legend 값
container name만 표시	{{name}}
Compose service만 표시	{{container_label_com_docker_compose_service}}
image 이름만 표시	{{image}}

예를 들어 query는 그대로 두고 Legend를 {{name}}으로 설정하면 차트 하단에는 observability-preview-grafana-1처럼 container name만 나온다. 이 lab에는 Dashboards > Paperclip Labs > W2D4 Observability Preview 대시보드가 provision되어 있으며, memory panel은 service 이름, CPU panel은 container name만 legend에 표시되도록 구성되어 있다.

curl로 Loki를 확인할 때는 시간 범위가 있는 endpoint를 사용한다. Grafana Explore는 화면의 time range를 함께 보내지만, API를 직접 호출하면 그 범위를 명시해야 한다.

WSL/Linux:

```
now_ns=$(date +%s%N)
start_ns=$((now_ns - 300000000000))

curl -G -s 'http://localhost:13100/loki/api/v1/query_range' \
  --data-urlencode 'query={job="docker"}' \
  --data-urlencode "start=$start_ns" \
  --data-urlencode "end=$now_ns" \
  --data-urlencode 'limit=5'
```

macOS:

```
end_time=$(date -u +"%Y-%m-%dT%H:%M:%SZ")
start_time=$(date -u -v-5M +"%Y-%m-%dT%H:%M:%SZ")
```

```
curl -G -s 'http://localhost:13100/loki/api/v1/query_range' \
  --data-urlencode 'query={job="docker"}' \
  --data-urlencode "start=$start_time" \
  --data-urlencode "end=$end_time" \
  --data-urlencode 'limit=5'
```

환경에 따라 Docker Desktop/WSL에서 Promtail이 /var/lib/docker/containers를 읽지 못할 수 있다. 이 경우에도 docker compose logs로 일반 로그를 확인하고, Grafana/Prometheus UI가 뜨는 것까지를 preview 성공으로 본다.

OS별 troubleshooting

환경	오류/증상	수업에서 잡는 포인트
WSL/Linux	docker-credential-desktop.exe not found	Docker CLI가 Windows credential helper 설정을 참조했지만 WSL PATH에서 실행 파일을 찾지 못한 상태
WSL/Linux	cAdvisor가 Docker data root를 못 읽음	docker info --format '{{.DockerRootDir}}'로 실제 DockerRootDir을 확인해야 함
WSL/Docker Desktop	mkdir /var/lib/docker: read-only file system	/var/lib/docker bind mount source를 만들 수 없는 환경. 기본 실행으로 돌아가고 host-mount profile은 선택 처리
WSL/Linux/macOS	Grafana에서 Prometheus 연결 실패	datasource URL을 localhost:19090이 아니라 http://prometheus:9090으로 설정
WSL/Linux/macOS	Post "http://localhost:19090/api/v1/query": connect: connection refused	Grafana가 아직 잘못된 URL을 보고 있음. Data source 편집 화면을 새로 열고 URL을 http://prometheus:9090으로 저장
WSL/Linux/macOS	port is already allocated	이미 실행 중인 container가 host port를 점유한 상태. docker ps로 충돌 port를 찾음
WSL/Linux/macOS	Loki instant query 오류	log query는 시간 범위가 필요하므로 query_range를 사용
macOS	date +%s%N이 동작하지 않음	BSD date와 GNU date 옵션 차이
macOS	Promtail/Loki 로그가 비어 있음	Docker Desktop은 Linux VM 안에서 engine이 돌아가므로 host에서 보는 경로와 engine 내부 log 경로가 다를 수 있음
macOS	cAdvisor device/mount 오류	Docker Desktop VM 보안/장치 mount 제약. 실패 자체를 runtime 차이 관찰 사례로 다룸

Cleanup

```
docker compose down
# dashboard/log data까지 reset할 때만
# docker compose down -v
```

구름 EXP 배움일기

- Up과 정상 응답을 구분한 순간
- logs/inspect/exec/stats 중 가장 유용했던 명령
- 내가 재현한 실패와 첫 확인 명령
- cleanup에서 지우면 안 된다고 판단한 것
- Day 5 Compose로 옮기면 편해질 option
- Kubernetes/Terraform에서 다시 만날 것 같은 설정 분리 질문

- `docker stats`와 Prometheus query가 어떻게 다르게 보였는가
- log와 metric 중 원인 힌트가 더 빨리 보인 쪽은 무엇인가
- Grafana 화면이 있어도 결국 어떤 `command/output`을 같이 봐야 하는가

핵심 포인트

Day 4의 마지막 목표는 dashboard를 잘 꾸미는 것이 아니다. 단일 container 명령으로 보던 증거가 Compose와 observability stack에서 metrics/logs로 확장된다는 감각을 잡는 것이다.